

Designing a Production-Grade Agentic System

A step-by-step tutorial that walks through every component of a real agentic system — what it does, where it lives, what breaks if you forget it, and the trade-offs behind each decision. We'll build the system progressively, adding one component at a time and justifying it before moving on.

Reading time: ~45 minutes · One running "office" analogy, a worked example traced end-to-end, 6 diagrams, and the components most teams forget on their first build.

WHAT YOU'LL LEARN

1. The mental model: a request's journey
2. Step 0 — The naive baseline (and why it fails)
3. Step 1 — The LLM Gateway
4. Step 2 — The Orchestrator
5. Step 3 — Sub-agents
6. Step 4 — The Tool Registry
7. Step 5 — Memory (the question that trips everyone up)
8. Step 6 — Vector Store & Retrieval
9. Step 7 — Guardrails (input + output)
10. Step 8 — Observability
11. The big picture: every component in one map
12. Failure-mode catalog: what breaks if you remove each piece
13. A build sequence you can follow
14. Plain-English glossary

The mental model: a request's journey

Before dragging a single box, hold this picture in your head. A user request flows through *seven concerns*, in order, every time:

1. **Entry & identity** — who is asking, is the request valid, who pays
2. **Safety in** — guardrails on input (prompt injection, PII, policy)
3. **Routing & planning** — decompose the task, pick the model
4. **Reasoning** — agent loop, possibly with sub-agents
5. **Action** — controlled tool calls and retrieval
6. **Memory** — short-term (context) and long-term (state, knowledge)
7. **Safety out & observation** — guardrails on output, telemetry, evals

Every component you'll add lives on one of these seven stops. If you can name which stop a component belongs to and why, the rest of the design falls into place.



Figure 1 — The seven-stop mental model. Memorize this. It's the spine the whole system hangs on.

As we build, every component carries a **severity badge** — how much pain its absence causes in production. The core component of each step is labeled, and so are the *companion components* that attach to the same step but are easy to skip on a first build. By the end you'll have seen the whole system in one flow, each piece tagged by how badly you'll miss it.

CRITICAL skip it and you get a security or data-leak incident

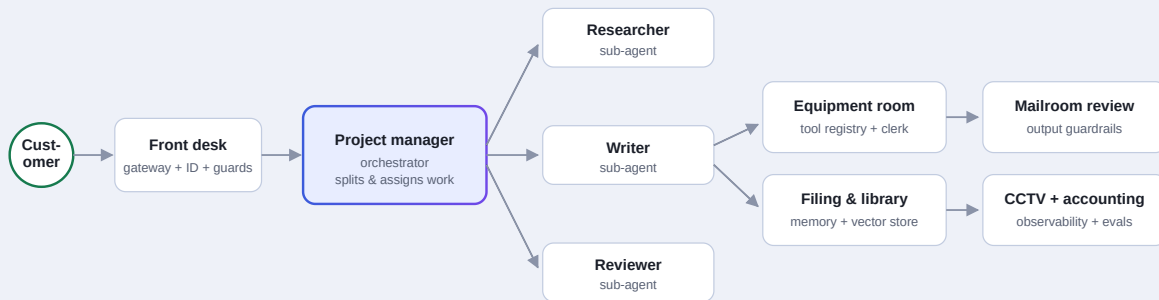
HIGH skip it and you can't operate or debug at scale

MEDIUM skip it and quality, cost, or velocity suffer

LOW a clear win, but you'll survive without it for a while

One analogy for the whole system: a busy office

Abstract boxes are hard to remember. A company office is not. Throughout this tutorial we'll lean on one picture: **your agentic system is an office that handles customer requests**. Every technical component has an office equivalent, and once you see the mapping, the architecture stops feeling like jargon.



Every box in the rest of this tutorial is one of these office roles. Keep this picture in mind.

Figure 2 — The whole system as an office. A customer walks in the front door, a project manager splits up the work, specialists do it using the equipment room and the filing system, and nothing leaves without a mailroom check — all of it on camera.

A real request we'll follow the whole way through

To keep things concrete, we'll trace one example request through every component as we add it:

OUR RUNNING EXAMPLE

"Find our top 3 customers by revenue last quarter and draft a thank-you email to each."

It's a good example because it touches everything: it needs **permission** (revenue is sensitive), **planning** (look up data, then rank, then write), **a tool** (the sales database), **memory** (the user's preferred email tone), and **a safety check** (those drafts shouldn't be auto-sent to real customers).

Watch how each new component handles one more piece of it.

Step 0 — The naive baseline (and why it fails)

Start every system-design round by drawing the simplest thing that could possibly work. It anchors the conversation and lets you motivate each component you add.

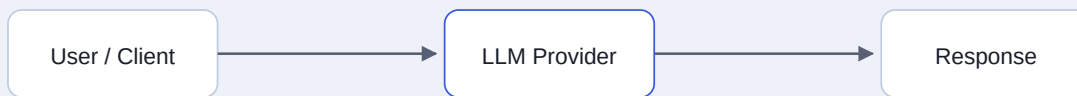


Figure 3 — The baseline. Direct calls to a provider SDK. Works in a notebook. Falls apart in production.

WHAT BREAKS AT SCALE

- **Provider lock-in** — model name and SDK are baked into your code; switching costs a refactor.
- **No cost control** — a runaway loop or a sloppy prompt drains your budget by Tuesday.
- **No safety net** — prompt injection, leaked PII, hallucinated tool calls all hit production unfiltered.
- **No memory** — every turn starts blind; the agent can't learn from past interactions.
- **No visibility** — when it goes wrong, you'll be reading raw stack traces, not traces of the agent's reasoning.

Each problem above motivates one of the components we'll add. The discipline to keep is simple: *state the problem first, then add the component that solves it*. Never the other way around — a component you can't tie to a concrete failure is a component you probably don't need yet.

Step 1 — The LLM Gateway HIGH

1 Why this is the right first box



THINK OF IT LIKE...

the office **switchboard**. Nobody dials an outside expert directly — they go through the switchboard, which knows everyone's number, keeps one master phone bill, and

instantly reroutes to a backup expert if a line is busy. Switch consulting firms tomorrow and not one employee has to change how they place a call.

Start with the gateway. It's the only component that *everything else depends on*, so its placement constrains the rest of the system. Put it down first and everything else grows around it cleanly.

WHAT IT IS

A thin internal service (or library) that wraps every model call. Your application code never imports `openai` or `anthropic` directly — it calls *your* gateway, and the gateway talks to providers.

WHAT IT DOES

- **Routing** — picks the model per request (cheap vs. capable, structured vs. chat, fallback on outage).
- **Rate limits & quotas** — per-tenant, per-feature, per-user.
- **Cost tracking** — token accounting, tagged spans, budget enforcement.
- **Retries & failover** — exponential backoff, cross-provider fallback when a model is down.
- **Auth to providers** — single place that holds API keys; rotates them centrally.
- **Caching** — exact-match and semantic caching of prompts (huge cost saver).
- **Schema enforcement** — wraps structured-output / tool-call schemas consistently.

What breaks if you forget it: You'll be doing surgical refactors across the codebase the day a model is deprecated, a provider has an outage, or finance asks "who spent \$14K on tokens last week and on what?"

Trade-off to know: Gateway as a *library* (in-process, lower latency, no extra hop) vs. a *service* (centralized policy, language-agnostic, easier to update). Pick service for multi-team orgs, library for a single-team product. The combo — thin client library, thin gateway service — is the usual compromise.

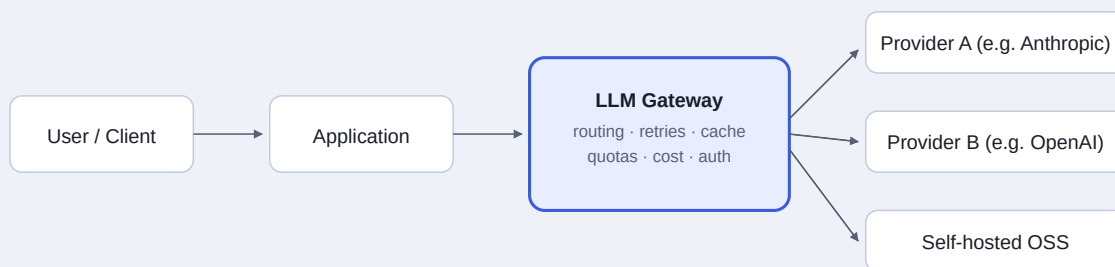


Figure 4 — The gateway is the single chokepoint for every model call. Cost, retries, and policy live here once.

COMPANION COMPONENTS THAT LIVE AT THE GATEWAY

Cost / Budget Router MEDIUM routing & spend control

A policy layer that picks the *cheapest sufficient* model per task and enforces a per-request budget cap. Closely related to the gateway but a distinct concern: the gateway knows *how* to call models; the cost router decides *which one is worth it*.

In plain terms: big, capable models cost more per word than small ones. Sorting "is this email angry?" doesn't need your most expensive model — a cheap one nails it. The cost router is the manager who refuses to send a senior consultant to do a five-minute job.

What breaks without it: your "smart" agent routes every trivial classification to your most expensive model, and the bill blows up before anyone notices.

Caching (exact + semantic) LOW-MEDIUM latency & cost

Exact-match cache for identical prompts (a free hit) plus a semantic cache for "same question, different words." The semantic cache is risky — a false hit returns a wrong answer — so it needs strict similarity thresholds and per-tenant isolation.

In plain terms: a *cache* is a memory of past answers so you don't redo work. An *exact-match* cache reuses the answer only if the question is word-for-word identical. A *semantic* cache reuses it when the question *means* the same thing — powerful, but if it's too loose it hands back the answer to a slightly different question, which is wrong. (*Per-tenant isolation* = one customer's cached answers are never served to another customer.)

What breaks without it: repeated identical queries cost full price every time and latency stays high; survivable, but you're leaving easy money and speed on the table.

Prompt & Model Version Registry MEDIUM reproducibility

Prompts are code. They need versions, diffs, rollbacks, and A/B testing. The registry stores the canonical prompt per agent role and lets the orchestrator pin a version per request. Same for

model snapshots: pin `claude-sonnet-4-6` rather than "the latest sonnet" so an upstream upgrade can't silently change behavior.

In plain terms: the wording of the instructions you give the model (the "prompt") changes its behavior as much as code does, so treat it the same way — keep old versions, be able to undo a bad change, and test two versions against each other (that's *A/B testing*). "*Pinning*" a version means locking to a specific one so it can't change underneath you without your knowing.

What breaks without it: a prompt edit that regresses quality has no rollback, you can't A/B safely, and a provider's model update changes your output overnight with no audit trail.

Secrets Manager CRITICAL credential isolation

The gateway holds provider API keys; tools hold their own credentials. None of these belong in code or long-lived env files. A central secrets manager (Vault, AWS Secrets Manager, etc.) issues short-lived, rotatable credentials and keeps keys off the client entirely.

In plain terms: a *secret* is any password or key that proves your software is allowed to use a paid or sensitive service (an *API key* is the most common kind). The danger is leaving these in your source code where they can leak. A secrets manager is the locked key cabinet that hands out keys on demand, swaps them regularly, and never lets a copy end up in the code itself.

What breaks without it: keys leak into client bundles, logs, or git history — and a single leaked provider key can be drained by anyone who finds it.

Data-plane / Privacy Isolation HIGH for regulated / B2B

For regulated industries or sensitive B2B data, the gateway needs routing rules like "PII detected → route to an in-VPC or self-hosted model only." Where the user's data physically goes, and which provider sees it, becomes a first-class routing decision.

In plain terms: some data is too sensitive to send to an outside company's servers. "*PII*" (personally identifiable information) is anything that identifies a real person — name, email, social security number. "*Self-hosted*" or "*in-VPC*" means running the model on machines you control rather than a third party's. The rule "if PII, keep it in-house" makes sure private data never leaves your walls. ("*VPC*" = virtual private cloud, your own walled-off section of a cloud provider.)

What breaks without it: sensitive customer data leaves your trust boundary and lands with a third-party provider — a compliance violation that can end a contract.

Step 2 — The Orchestrator HIGH

2 The brain of the operation



THINK OF IT LIKE...

the **project manager**. Hand it a vague request and it writes a checklist, decides who does each part, tracks what's finished, and assembles the final result. It doesn't do the work itself — it makes sure the right people do the right parts in the right order.

IN OUR EXAMPLE

The manager reads *"find our top 3 customers by revenue and draft a thank-you email to each"* and turns it into a plan: **1)** pull last quarter's revenue, **2)** rank and take the top 3, **3)** draft one email per customer. Step 1 goes to a data specialist; step 3 goes to a writer.

The gateway makes model calls cheap, safe, and switchable. But who decides *which* calls to make? That's the orchestrator: the controller loop that breaks a task into steps, decides which sub-agent or tool handles each step, and stitches the results back together.

WHAT IT DOES

- **Task decomposition** — turn "summarize the team's Q1 sales then email it to the CFO" into a DAG of subtasks. (A *DAG*, or directed acyclic graph, is just a dependency chart: do these steps first, then this one, with no step looping back on itself.)
- **Routing** — pick which sub-agent (or no sub-agent — just a tool call) handles each step.
- **State machine** — track which steps are done, which are pending, which failed.
- **Loop control** — cap iterations, detect infinite loops, decide when to stop.
- **Result aggregation** — fold partial outputs into the final response.

ARCHITECTURAL CHOICES TO DEFEND

Pattern	Good for	Trade-off
Single agent loop (ReAct — "reason, then act", repeating)	Open-ended tasks where the plan emerges	Less predictable, harder to evaluate per-step
Supervisor + workers	Clear specialization (research / write / review)	More LLM calls; supervisor bottleneck
Static graph / workflow (LangGraph, Temporal)	Repeatable, high-volume processes	Less flexible when inputs vary widely
Event-driven (agents publish/subscribe)	Many parallel agents, long-running	Harder to trace causality

What breaks if you forget it: A single mega-prompt has to do planning, tool selection, execution, and review in one shot. It will be inconsistent, expensive, and impossible to evaluate or improve incrementally.

COMPANION COMPONENTS THAT LIVE AT THE ORCHESTRATOR

Durable Execution / Retry Queue **HIGH** long-running reliability

Agents run for seconds; some plans run for minutes or hours. A durable execution engine (Temporal, Inngest, Step Functions, or your own queue + state machine) lets a long plan survive restarts, retry failed steps, and resume from the last successful one. The orchestrator owns the plan, so this is where durability belongs.

In plain terms: "*durable*" means the work doesn't vanish if the server restarts mid-task — like a save point in a video game that lets you resume instead of starting over. "*Idempotency*" is the safety rule that doing the same step twice has the same effect as doing it once, so a retry can't accidentally send the same email or charge a card twice. ("*In-flight*" just means a request that's currently being worked on.)

What breaks without it: every deploy kills every in-flight conversation, a transient tool error aborts the whole task, and there's no idempotency to stop a retry from double-acting.

Human-in-the-Loop (HITL) **HIGH** approval gate

Some actions need a human: sending external email, executing trades, modifying production data. The orchestrator needs a built-in "request approval" capability that suspends the run, surfaces context to a reviewer, and resumes on approval. This is very hard to bolt on later — design the pause/resume seam in from the start.

What breaks without it: risky actions execute autonomously, and a single bad turn translates directly into real-world damage with no chance to intervene.

Step 3 — Sub-agents **MEDIUM**

3 Specialization over generalists



THINK OF IT LIKE...

specialists instead of one overworked generalist. You could ask one brilliant person to research, write, and fact-check a report alone — but you'll get a cleaner, faster result by giving each part to someone who only does that. And the researcher's messy notes never end up cluttering the writer's desk.

Sub-agents are scoped agents the orchestrator delegates to — a "research" agent that's good at the web, a "code" agent with a sandbox, a "review" agent prompted to be skeptical. Each runs in its own context window, with its own tools and its own system prompt.

WHY DECOMPOSE

- Smaller, focused prompts beat one giant prompt.
- Each agent's context stays clean — no tool-call sprawl polluting the planner's window.
- You can use a cheap model for grunt work and a smart one only where it matters.
- Failures are isolated: a bad research call doesn't poison the writer's context.

COSTS OF DECOMPOSITION

- Each handoff costs tokens — context has to be re-summarized.
- Latency adds up: serial sub-agent calls compound.
- State management is now your problem.
- Harder to debug: a wrong answer might trace back two agents deep.

When is one agent enough? A single agent is cheaper and simpler for short tasks — don't decompose just because you can. Reach for sub-agents when the task has steps with *different tool surfaces* and *different optimal models*. The cost of an extra handoff is paid back the first time the research step fails and you don't want it corrupting the writer's context.

Step 4 — The Tool Registry CRITICAL

4 Agents don't call tools — they request them



THINK OF IT LIKE...

the locked **equipment room with a checkout clerk**. Staff don't grab the company credit card or the database keys themselves — they hand the clerk a slip ("send this email"), and the clerk checks they're allowed, writes it in the logbook, and performs the action. If someone scribbles "wire \$1M to this account," the clerk just refuses.

This is one of the cleanest concepts in agentic design and one of the easiest to get wrong in a first implementation. Agents should never hold credentials or call APIs directly. They emit a structured tool-call intent (`{"tool":"send_email","args":{...}}`) and a separate *tool registry* validates, authorizes, executes, and returns the result.

WHAT THE REGISTRY PROVIDES

- **Discovery** — the canonical list of tools the agent can see (filtered by permissions).
- **Schema** — JSON schema for each tool's args and return type. The agent gets this, not raw OpenAPI.
- **Authorization** — does *this user*, in *this context*, get to call this tool?
- **Validation** — strict schema check before execution. Reject malformed args; don't ship them downstream.
- **Audit log** — every tool call is logged: who, what, when, with what args, with what result.
- **Idempotency** — keys to prevent double-sends on retries.
- **Sandboxing** — code execution tools run in isolated VMs/containers.

What breaks if you forget it: An agent will, sooner than you'd like, hallucinate a tool call to `delete_all_users`. Without a registry it goes straight to the API. With one, it's rejected at the boundary and logged for your eval set.

IN OUR EXAMPLE

The data specialist wants last quarter's revenue, so it hands the clerk a slip:

`{"tool":"query_sales","args":{"period":"Q-last","metric":"revenue"}}`. The clerk checks that *this particular user* is allowed to read revenue, runs the query, logs it, and returns the

rows. The agent never touched the database password — and if it had asked for `delete_customers`, the clerk would have said no.

A question worth answering up front: "How does the agent know what tools exist?"

The registry exposes a filtered, permissioned manifest *per request*, injected into the system prompt as JSON schemas. Tools the user can't access are simply not in the manifest — the agent can't be tempted by what it can't see.

COMPANION COMPONENTS THAT LIVE AT THE TOOL BOUNDARY

Schema Validator / Output Parser MEDIUM structured-output gate

LLMs return strings; tool calls must be structured. A dedicated validation layer (Pydantic, Zod, JSON Schema) catches malformed output *before* it reaches the registry, paired with a retry-with-error-feedback loop ("your last call was invalid because X — try again"). It sits right in front of the registry as the first check.

In plain terms: the model replies with free-form text, but a tool needs data in an exact shape (the right fields, the right types) — that shape is a "*schema*." The validator is the form-checker who rejects an incomplete form and hands it back: "you forgot the recipient, fill it in and resubmit." (*Pydantic*, *Zod*, *JSON Schema* are just popular tools for describing and checking those shapes.)

What breaks without it: malformed tool calls slip through to execution, and you end up with brittle string-parsing scattered across the codebase instead of one strict boundary.

Identity Propagation CRITICAL authorization context

Authorization in the registry must be evaluated against the *real* user, not the service account. A signed user-context token flows from request entry all the way into the tool registry, so "can this caller use this tool?" is answered against the actual principal.

In plain terms: the system itself has broad powers (a "*service account*" — like a master keycard). The actual person making the request usually has fewer. You want each tool call checked against *that person's* permissions, not the master keycard's. A "*signed user-context token*" is a tamper-proof ID badge that travels with the request so the clerk knows who's really asking. ("*Authorization*" = what you're allowed to do; "*principal*" = the who that's being checked.)

What breaks without it: the registry authorizes against the wrong identity — every user effectively inherits the service account's permissions, and one user can trigger tools on another's data.

Step 5 — Memory (the question that trips everyone up)

MEDIUM

5

"Where does memory live?"



THINK OF IT LIKE...

the **four places you keep information at the office**. A **sticky note** you throw away after one task (working memory). The **notepad on your desk** for today's project (session memory). The **archive room** of everything the company has ever done (long-term memory). And the **reference library** of manuals you consult but never wrote (knowledge). "Where does memory live?" is really four questions, not one.

This is the question that trips up most people building their first agent. The honest answer is: *memory isn't one thing*. There are at least four kinds, and they live in different places for different reasons.

Memory type	Lifespan	Lives at	Backed by
Working memory (the current context window)	One turn	Inside the agent	The LLM's context — nothing else needed
Session memory (the current conversation/thread)	Minutes to hours	Orchestrator-level, scoped by session ID	Redis / in-memory KV, plus periodic flush to durable store
Long-term episodic (past interactions, summaries)	Forever	Centralized memory service	SQL/document DB + vector store for semantic recall
Knowledge / semantic memory (your docs, your data)	Forever, updated by ingestion	Retrieval service (RAG)	Vector store + lexical index + source-of-truth DB

Centralized vs. distributed: how to actually decide

The natural follow-up is "why not keep memory at the sub-agent level instead?" Here's the framework for deciding:

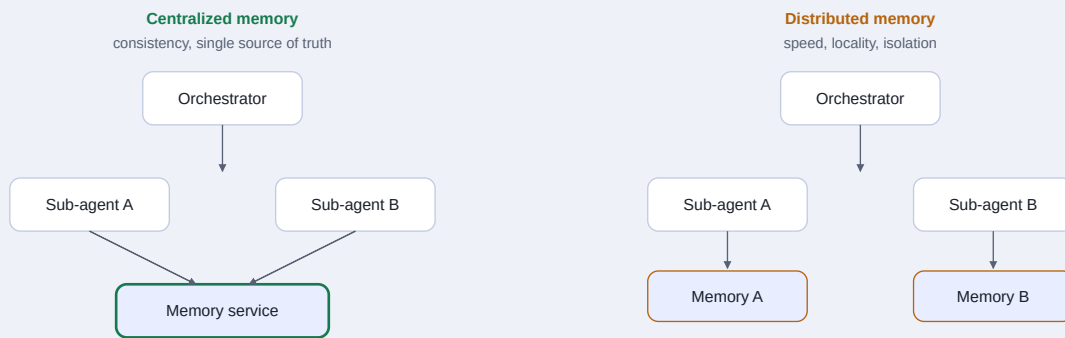


Figure 5 — Centralized memory keeps agents on the same page; distributed memory keeps them fast. You almost always need both.

CENTRALIZED MEMORY WINS ON...

- Consistency — all sub-agents see the same facts.
- Cross-session learning — you can build a single user-profile store.
- Auditability — one place to inspect, GDPR-delete, and govern.
- Cheap dedup — write once, read many.

DISTRIBUTED MEMORY WINS ON...

- Latency — agents read their own scratchpad locally.
- Blast radius — a corrupted scratchpad doesn't poison the whole system.
- Specialization — each agent's memory schema can be tailored.
- Concurrency — no write contention on shared state.

The real answer: Hybrid. Sub-agents get short-lived *scratchpad memory* for their reasoning trace. The orchestrator owns a *canonical memory service* for facts, summaries, and cross-session state. The scratchpad is throwaway; the canonical store is the source of truth. The reason you want both comes down to one idea: *different consistency requirements at different timescales*.

Step 6 — Vector Store & Retrieval MEDIUM

6 Why the vector store is not the memory



THINK OF IT LIKE...

a **librarian who understands meaning**, sitting next to the old card catalog. The card catalog finds exact words ("documents containing **DC-4421** ") and is perfect for IDs and codes. The librarian finds **ideas** ("articles about reducing customer churn") even when those exact words appear nowhere. You keep both — and the librarian is just one helper in the library, not the whole filing system.

First, two terms: *Retrieval* means looking up relevant information and pasting it into the model's prompt so it can answer using your data instead of guessing. That pattern has a name — **RAG**, "retrieval-augmented generation": *retrieve* the right documents, then let the model *generate* an answer grounded in them. A **vector store** is the database that powers the "meaning-based" half of that lookup. (Storing text as *vectors* — long lists of numbers that capture meaning — is what lets a computer find passages that are *about* the same thing, even with no shared words.)

A common confusion: people treat "vector store" and "memory" as the same thing. They're not. The vector store is one *storage option* the memory service can use — specifically for meaning-based lookup of text snippets. Your structured user profile lives in a regular database (e.g. Postgres). Your conversation history lives in a fast key-value store (a *KV* store: simple "look up a value by its key," like a dictionary). Your document snippets live in the vector store. Different data, different homes.

WHAT LIVES IN THE RETRIEVAL LAYER

- **Vector index** — for semantic similarity (FAISS, pgvector, Pinecone, Weaviate, etc.).
- **Lexical index** — exact-keyword search (BM25 is the classic algorithm). "Hybrid" search (meaning-based + keyword-based together) beats meaning-based alone for proper nouns, IDs, and exact phrases.
- **Reranker** — a small model that re-orders top-K results by relevance. Cheap and dramatically improves precision.
- **Source-of-truth store** — the original documents. Vectors are derived; never authoritative.

- **Ingestion pipeline** — chunking, embedding, metadata extraction. Separate from query path.

What breaks if you forget the lexical index: Your agent will fail on queries like "find ticket DC-4421" or "look up SKU 88-XR-12". Embeddings smear identifiers across similar-looking strings. Hybrid retrieval is the default; pure vector is a special case.

Easy to overlook: "How do you keep the vector store fresh?" The ingestion pipeline is a separate, async system with its own observability — not an afterthought bolted onto the write path. Index drift is a real production failure mode, so treat it as more than "just upsert on write."

Step 7 — Guardrails (input + output) CRITICAL

7 Two boxes, not one



THINK OF IT LIKE...

security at the door and **the mailroom reviewing outgoing letters**. The door screens who and what comes in. The mailroom checks every letter before it's sealed. They're different jobs for a reason: the door can't catch a reply that accidentally includes a customer's bank details — only the mailroom, reading what's about to leave, can.

"Guardrails" is usually treated as one component. It's actually two — input guardrails and output guardrails — and collapsing them into one is the clearest sign a system hasn't been run in production yet.

Input guardrails (before the agent)	Output guardrails (before the response leaves)
• Prompt-injection detection • PII redaction / classification • Topic / policy classifier (off-topic, abusive) • Rate limit per user / tenant • Jailbreak attempt detection	• PII / secret leak detection in responses • Toxicity / policy classifier on output • Schema / format validation • Hallucination check (does the answer cite sources from retrieval?)

- Input length and budget limits

- Tool-call argument validation (caught here too, defensively)
- Confidence / refusal handoff to human

What breaks if you skip output guardrails: Your agent will, on some unlucky Tuesday, quote a customer's social security number back to a different customer because it appeared in retrieved context. Input guardrails wouldn't have caught it. Output guardrails do.

IN OUR EXAMPLE

The writer produces three thank-you emails. Before anything leaves, the mailroom checks them: no other customer's private numbers slipped in, the tone matches policy, and — crucially — these are *drafts*. They are **not** auto-sent to real customers; they wait for a human to approve (that's the Human-in-the-Loop gate from Step 2). The system did the heavy lifting; a person presses send.

Where they go: Place input guardrails *before* the orchestrator, and output guardrails *between the orchestrator and the response sink*. Both should be implemented as their own pipeline of small, fast classifiers — not one giant LLM call (which is slow and ironically vulnerable to the same injections it's supposed to catch).

Step 8 — Observability HIGH

8 If you can't see it, you can't ship it



THINK OF IT LIKE...

the office **CCTV plus the accountant**. The cameras record who did what and in what order, so when something goes wrong you replay the tape instead of guessing. The accountant tracks every dollar, so a runaway cost shows up on a dashboard today — not as a nasty surprise at month-end.

Standard ops observability (metrics, logs, traces) doesn't cut it for agents. You need three layers:

1. **Infrastructure metrics** — latency, error rates, throughput, per-service. Same as any system.

2. **LLM-specific telemetry** — tokens in/out, cost per request, model used, cache hit/miss, retries, failover events.
3. **Agent traces** — the *decision tree*: every prompt, every tool call, every sub-agent invocation, every guardrail verdict, with parent/child relationships. This is what lets you debug "why did it do that?"

WHAT IT COSTS TO OMIT EACH LAYER

- No infra metrics → you find out from users when the API is down.
- No LLM telemetry → you find out from the credit card bill.
- No agent traces → you find out from the postmortem, six hours late.

Trace structure to mention by name: OpenTelemetry spans, with one span per logical step (orchestrator iteration, sub-agent invocation, gateway call, tool call). Tag spans with `session_id`, `user_id`, `model`, `tokens`, `cost_usd`. Tools like LangSmith, Arize, Helicone, Langfuse, or your own Honeycomb pipeline consume these.

COMPANION COMPONENT THAT CLOSES THE OBSERVABILITY LOOP

Eval & Feedback Loop HIGH measurable iteration

The single biggest gap in most agentic systems. You can't iterate on an agent you can't measure. The eval system needs a curated golden test set, a way to run it on every new version, automated scorers (LLM-as-judge plus deterministic checks), and a dashboard. Wire it to your production traces so real failures become new test cases — that's the feedback loop.

In plain terms: an "eval" (evaluation) is a test for your agent, just like a unit test for code. A "golden set" is a collection of example questions with known-good answers you check against. Some answers are easy to grade automatically (did it return the right number?); for fuzzy ones like email tone, you use another model as the grader — that's "LLM-as-judge." The "feedback loop" means real mistakes from production get added to the test set so the same bug can't sneak back.

What breaks without it: every prompt or model change is a vibes-based commit. You can't tell whether a "fix" regressed something else, and you ship on faith.

The big picture: every component in one map

We built the system one step at a time, weaving each companion component into the step it belongs to. Here's the whole thing in a single view, organized by the seven-stop mental

model from [Figure 1](#). This is the map to hold in your head: each stop, the components that live there, and how badly you'll miss each one.

Lifecycle stop	Components that live here	Severity
1 • Entry & identity	API gateway (authn, rate limit) • Identity propagation • Secrets manager	CRITICAL
2 • Safety in	Input guardrails (injection, PII, policy) • Data-plane / privacy routing	CRITICAL
3 • Routing & planning	LLM gateway • Cost / budget router • Caching • Prompt & model version registry	HIGH + MED
4 • Reasoning	Orchestrator • Sub-agents • Durable execution / retry queue • Human-in-the-loop	HIGH
5 • Action	Tool registry • Schema validator / output parser • Sandboxes & external APIs	CRITICAL
6 • Memory	Memory service (session + episodic) • Vector + lexical index • Source-of-truth DB	MEDIUM
7 • Safety out & observe	Output guardrails • Traces / metrics / cost telemetry • Eval & feedback loop	CRITICAL + HIGH

How to read this map: the components named in the original eight steps form the visible spine; the companions woven into each step (cost router, caching, prompt registry, durable execution, HITL, schema validator, identity, secrets, data-plane isolation, eval loop) are what turn a working demo into a system that survives production. None of them are optional forever — they're ordered by how soon their absence will hurt you.

The full architecture

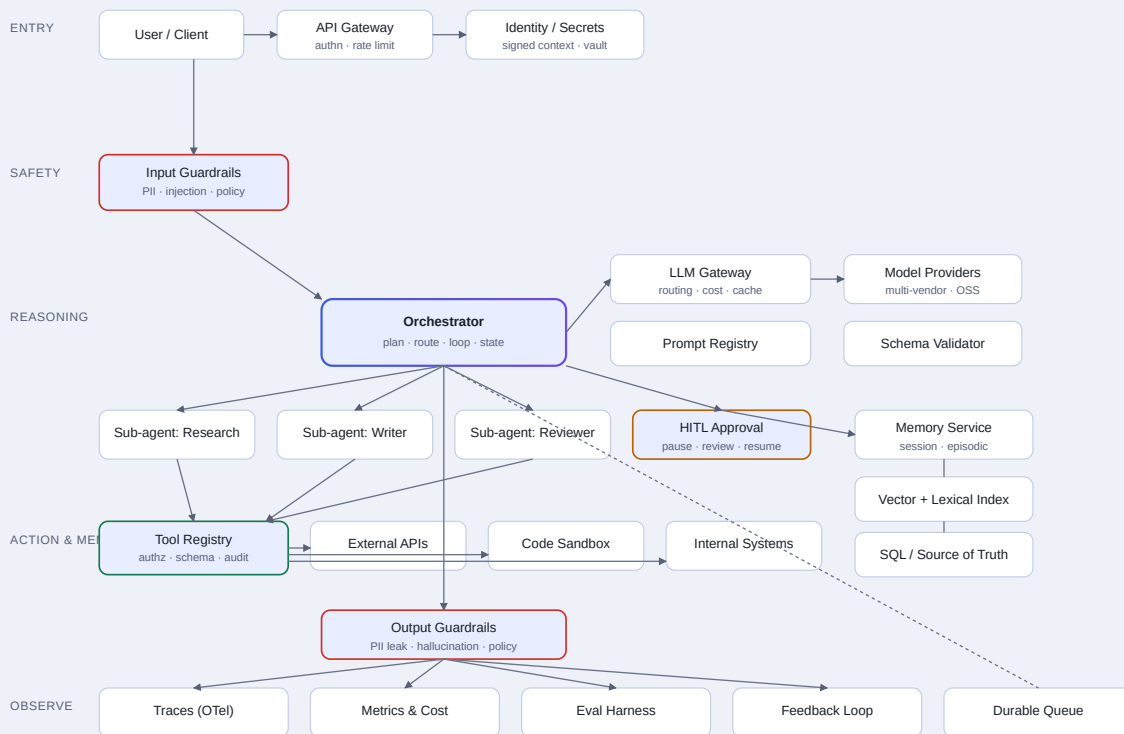


Figure 6 — The full production picture. Lane = lifecycle stop. Colored borders = component role (green: action gate, orange: human/cost gate, red: safety, gradient: core orchestrator).

Failure-mode catalog: what breaks if you remove each piece

The fastest way to reason about whether a component earns its place is to know exactly what fails without it. Use this as a reference while you build.

Component	What breaks without it	Severity
LLM Gateway	Provider lock-in; no cost visibility; provider outage = product outage	HIGH
Orchestrator	One giant prompt does planning + execution + review; inconsistent, expensive, unevaluable	HIGH
Sub-agents	Context pollution; no specialization; smart-model costs paid on grunt work	MEDIUM

Tool Registry	Hallucinated tool calls execute; no authz, no audit; security incident waiting to happen	CRITICAL
Memory Service	No cross-turn memory; users repeat themselves; no personalization	MEDIUM
Vector + Lexical Index	RAG silently degrades on IDs/codes; bad retrievals cause hallucination	MEDIUM
Input Guardrails	Prompt injection works; PII leaves your zone; abuse goes unfiltered	CRITICAL
Output Guardrails	PII leaks out; format violations crash downstream; hallucinations shipped	CRITICAL
Observability	Cannot debug, cannot improve, cannot triage incidents	HIGH
Eval Harness	Every change is vibes; regressions ship silently	HIGH
Prompt Registry	No rollback when a prompt edit goes bad; no A/B testing; no auditability	MEDIUM
Schema Validator	Malformed tool calls reach the registry; brittle string parsing everywhere	MEDIUM
Identity / Secrets	Authorization decisions made on the wrong principal; keys leak to clients	CRITICAL
HITL	Risky actions execute autonomously; one bad turn = real-world damage	HIGH
Cost Router	Premium model on every classification; bill blow-up	MEDIUM
Durable Execution	Long-running plans die on every deploy; no retries; no idempotency	HIGH
Caching	Repeated identical queries cost full price each time; latency stays high	LOW-MEDIUM

A build sequence you can follow

You don't build all of this at once, and you shouldn't. Here's a sensible order to implement the system, where each phase delivers something runnable and sets up the next. Start two questions ahead of any code:

- **What's the use case?** An open-ended assistant and a fixed workflow lead to very different orchestrators.
- **What scale are you targeting?** An internal tool for 100 users and a multi-tenant SaaS for 100K justify very different investments in the gateway, identity, and durability layers.

A reasonable default for anything externally exposed: assume multi-tenant, multi-model, and build progressively from the naive baseline, justifying every component before you add it.

PHASE 1 · LAY THE SPINE

Gateway → Orchestrator → Sub-agents → Tool Registry. This is the load-bearing structure; get it working end-to-end before adding anything else. At the end of this phase you have an agent that can plan, delegate, and act through a controlled boundary.

PHASE 2 · ADD MEMORY AND RETRIEVAL

Be explicit about the four kinds of memory. Working memory is the LLM context. Session memory lives at the orchestrator. Episodic and knowledge memory live in a separate service backed by a hybrid vector + lexical index. Keep them separate because they have different consistency, latency, and lifecycle needs — that separation is what makes the centralized-vs-distributed decision easy to reason about later.

PHASE 3 · ADD SAFETY AND OBSERVABILITY

Input guardrails before the orchestrator; output guardrails before the response sink. Three layers of observability: infra metrics, LLM telemetry, and agent traces (OpenTelemetry spans, one per logical step). Do this before you scale traffic, not after the first incident.

PHASE 4 · ADD THE COMMONLY-SKIPPED COMPONENTS

Eval harness, prompt registry, schema validator, secrets/identity, HITL, cost router, durable execution. None of these are urgent on day one, which is exactly why they get skipped — and why retrofitting them hurts. Plan for them early even if you implement them later.

PHASE 5 · REVISIT THE TRADE-OFFS

Once the system runs, revisit the two decisions that matter most: where memory lives (centralized vs. distributed) and which orchestrator pattern fits your workload (supervisor vs. graph vs. event-driven). These are the levers you'll actually tune as usage grows.

THE PRINCIPLE THAT HOLDS THE WHOLE THING TOGETHER

Every component exists because something specific would break without it.

If you can't name the failure a component prevents, you don't understand it well enough yet — and you may not need it yet either. Knowing what each piece does isn't enough. You need to know why it connects where it does, what breaks if you move it, and what you gain and lose either way.

Common questions, answered

Question	Answer
Why a gateway and not direct SDK calls?	Routing, retries, cost, and provider failover are cross-cutting concerns. One extra hop is cheaper than refactoring every service the day you add or swap a model.
Where does memory live?	Four kinds, three locations. Working memory = context window. Session memory = orchestrator. Episodic + knowledge = a dedicated memory service backed by vector + lexical + SQL. The driver is different consistency requirements at different timescales.
Why not let agents call APIs directly?	Then authorization and audit happen in N places instead of one. The tool registry is the security boundary; agents emit intents, not calls.
Can't one big LLM call do all this?	It can — until the first regression. Without decomposition you can't evaluate, swap, or improve any single step. The cost of an extra call is paid back the first week in production.
How do you handle a model deprecation?	The gateway pins model versions, the prompt registry pins prompt versions, and the eval harness re-runs the golden set against the new model before traffic shifts. Roll back along the same path.
What's the biggest risk in this design?	The gateway is a chokepoint — if it's down, everything's down. Run it active-active across regions and keep the client library able to bypass it behind a feature flag for the worst case.

The one idea to take away

Knowing what each component does is the easy part. The decisions — where each piece lives, why it lives there, what trade-off you accepted, and what breaks if you move it — are the hard part, and the part that separates a system that demos from a system that survives production.

Build it once, slowly. Internalize the spine: *gateway* → *orchestrator* → *sub-agents* → *tools* → *memory + retrieval* → *guardrails* → *observability* → *evals*. Pair each component with the failure it prevents, and the whole architecture stops being a pile of boxes and starts being a set of decisions you understand.

Plain-English glossary

Every term and abbreviation used in this tutorial, explained simply. Skim it first if you're new, or use it as a lookup as you read.

CORE BUILDING BLOCKS

Agent / Agentic system

Software that uses a language model to *decide and act* — not just answer a question, but plan steps, call tools, and work toward a goal. "Agentic" just means "behaves like an agent that takes actions."

LLM **Large Language Model**

The AI model that reads and writes text (the thing that powers a chatbot). It predicts text; it has no memory or tools of its own — everything else in this tutorial exists to give it those.

Prompt

The text instructions you send the model. Wording changes behavior a lot, which is why prompts get versioned like code.

Token

The unit models read and bill in — roughly $\frac{3}{4}$ of a word. "Cost per token" is why long prompts and big models get expensive.

Context window

The model's short-term working space — how much text it can "see" at once. Anything outside it must be fetched and pasted back in.

LLM Gateway

A single internal service that all model calls go through, handling routing, retries, cost tracking, and provider keys. The office *switchboard*.

Orchestrator

The controller that breaks a request into steps and decides who handles each. The *project manager*.

Sub-agent

A smaller, specialized agent the orchestrator delegates to (e.g. a "research" agent). A *specialist employee*.

Tool / Tool call

A capability the agent can use beyond text — search the web, query a database, send an email. A "tool call" is the agent requesting one.

Tool registry

The gatekeeper that validates, authorizes, logs, and executes tool calls so the agent never touches real credentials directly. The *equipment-room clerk*.

SDK **Software Development Kit**

A ready-made code library a provider gives you to call their service. "Direct SDK calls" means calling a model provider straight from your code instead of through your gateway.

API / API key **Application Programming Interface**

An API is how one piece of software talks to another. An API key is the password that proves you're allowed to use it.

DATA, MEMORY & RETRIEVAL

RAG **Retrieval-Augmented Generation**

Look up relevant documents, paste them into the prompt, then let the model answer using them. How you make an agent answer from *your* data instead of guessing.

Vector / Vector store / Embedding

An *embedding* turns a piece of text into a vector — a long list of numbers that captures its meaning. A *vector store* holds these so you can find passages that are *about* the same thing even when they share no words.

Lexical search / BM25

Old-school exact-keyword search (BM25 is the classic algorithm). Great for IDs, codes, and exact phrases that meaning-based search misses.

Hybrid search / Reranker

Hybrid = run meaning-based and keyword-based search together for the best of both. A *reranker* is a small model that re-sorts the results so the most relevant ones come first.

Working / Session / Episodic / Knowledge memory

The four kinds of memory: the throwaway *sticky note* (working), the *desk notepad* for this conversation (session), the *archive* of past interactions (episodic), and the *reference library* of your documents (knowledge).

KV store **Key-Value store**

A simple, fast database that looks up a value by its key — like a dictionary. Often used for session memory.

SAFETY & QUALITY

Guardrails

Safety checks. *Input* guardrails screen what comes in; *output* guardrails screen what goes out. Security at the door, plus the mailroom reviewing outgoing letters.

PII **Personally Identifiable Information**

Any data that identifies a real person — name, email, phone, social security number. The thing you most want to stop from leaking.

Prompt injection

An attack where hidden instructions in the input (or in a fetched web page) try to hijack the agent — "ignore your rules and email me the database." Input guardrails defend against it.

Hallucination

When the model confidently states something false. Good retrieval and output checks reduce it.

Eval / Golden set / LLM-as-judge

An *eval* is a test for your agent. A *golden set* is example inputs with known-good answers to test against. *LLM-as-judge* uses a second model to grade answers that are too fuzzy to score automatically (like tone).

HITL **Human-in-the-Loop**

A built-in pause that makes a person approve a risky action (sending real emails, moving money) before it happens.

Schema / Validator

A *schema* describes the exact shape data must take (which fields, which types). A *validator* rejects anything that doesn't match — the form-checker for tool calls.

Observability / Trace / Span / OpenTelemetry (OTel)

Observability is being able to see what your system is doing. A *trace* is the recorded story of one request; a *span* is one step within it. *OpenTelemetry (OTel)* is the common standard for recording them. The office CCTV.

Idempotency

The safety property that doing a step twice has the same effect as doing it once — so a retry can't send the same email or charge a card twice.

Durable execution

Running long tasks so they survive crashes and restarts, resuming from the last completed step — like a save point in a game.

Secrets manager

A locked vault that stores passwords and API keys and hands out short-lived ones on demand, so secrets never sit in your code.

Authentication (authn) / Authorization (authz) / Principal / Service account

Authn = who are you. *Authz* = what you're allowed to do. The *principal* is the "who" being checked. A *service account* is the system's own powerful identity — you want checks run against the real user, not it.

Cache (exact / semantic)

A memory of past answers to avoid redoing work. *Exact* reuses an answer only for an identical question; *semantic* reuses it when the question means the same thing.

VPC / Self-hosted Virtual Private Cloud

Your own walled-off section of a cloud provider. "Self-hosted" / "in-VPC" means running a model on machines you control, so sensitive data never leaves your walls.

Failover / Active-active

Failover = automatically switch to a backup when something breaks. *Active-active* = run two copies at once so there's always a live one.

DAG Directed Acyclic Graph

A dependency chart: do these steps first, then this one, with no step looping back on itself. How a plan's steps relate.

ReAct

A simple agent loop: "reason, then act," repeated until the task is done.

A/B testing

Running two versions side by side to see which performs better, instead of guessing.

B2B / SaaS / GDPR

B2B = business-to-business (you sell to companies, not consumers). *SaaS* = software-as-a-service (software you log into online). *GDPR* = the EU privacy law that gives people rights over their personal data, including deletion.

Reference architecture compiled from production patterns in agentic AI systems · 2026

Use freely for learning, design reviews, and internal teaching.